Distributed Systems and Cloud Computing

00016577

Rajabboev Anvarmirzo

Table Of Contents

1 Application Overview

2 Database

3 **Containerization**

4. Docker compose -service configuration

5. Deployment configuration

6. CI/CD pipeline

7. SECRETS Management

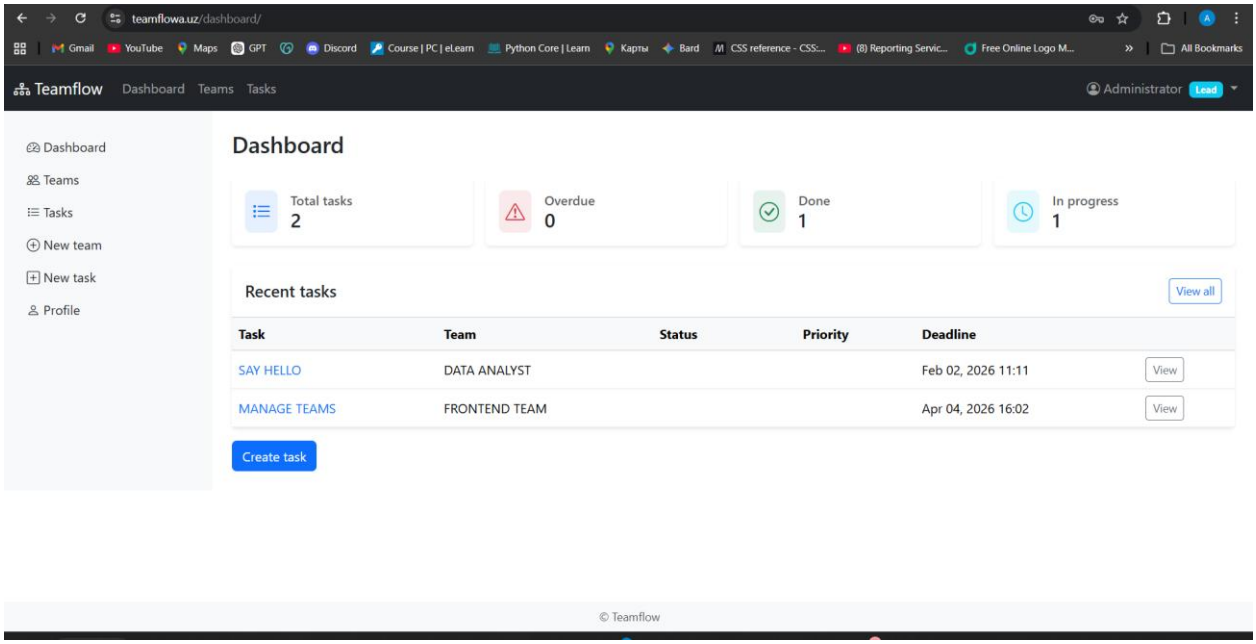
8. Challenges

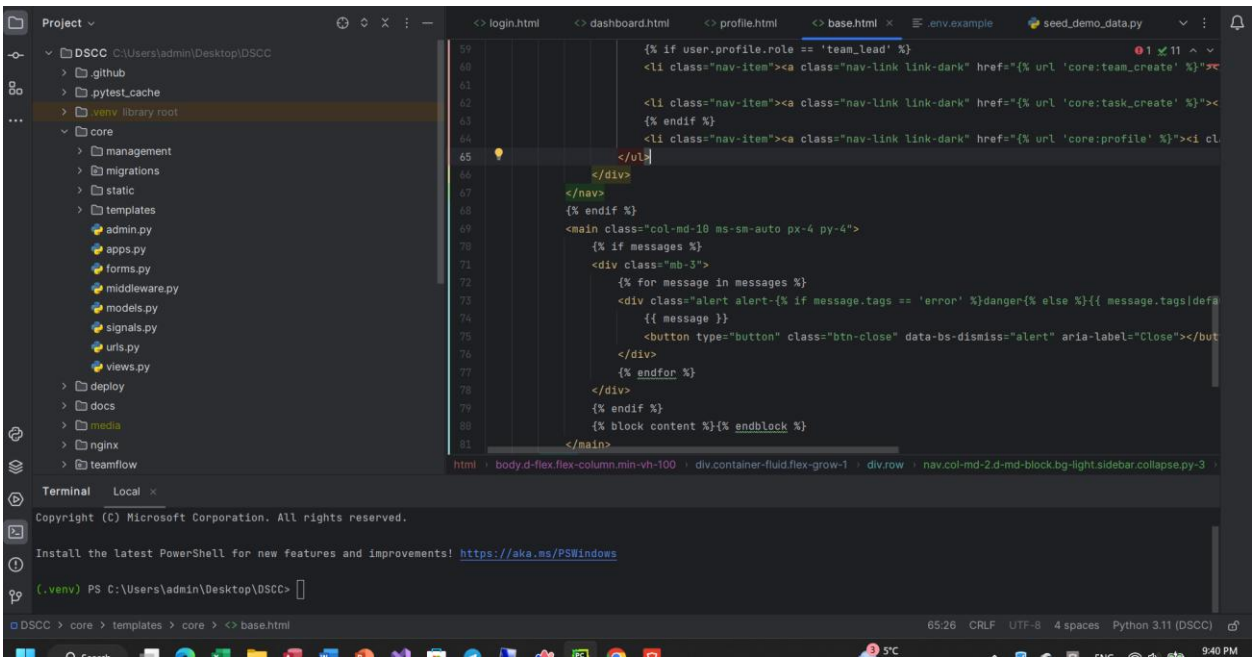
9. Future Improvements

10. Links

Application Overview

Team Flow by its name this project is built with Django for automating and managing collaborative tasks and teams. In this app users can register, log in and manage teams and tasks, there are two types of users ‘team leads’ and ‘members. Core technologies used in this project: Django as a web framework, Bootstrap for frontend, PostgreSQL as a database and Nginx as a reverse proxy, Gunicorn as the WSGI serving the application in production. The application runs inside Docker containers orchestrated with Docker Compose on a Google Cloud Compute Engine VM. In terms of the system, there are 3 primary database models Users, Teams, and Tasks which have both many-to-one and many-to-many relationships. Key features include user auth and CRUD operations for task management, different panels for admins (team lead) and members(user) and five functional views like dashboard, team management, admin panel, task create and feedback with task tracking.

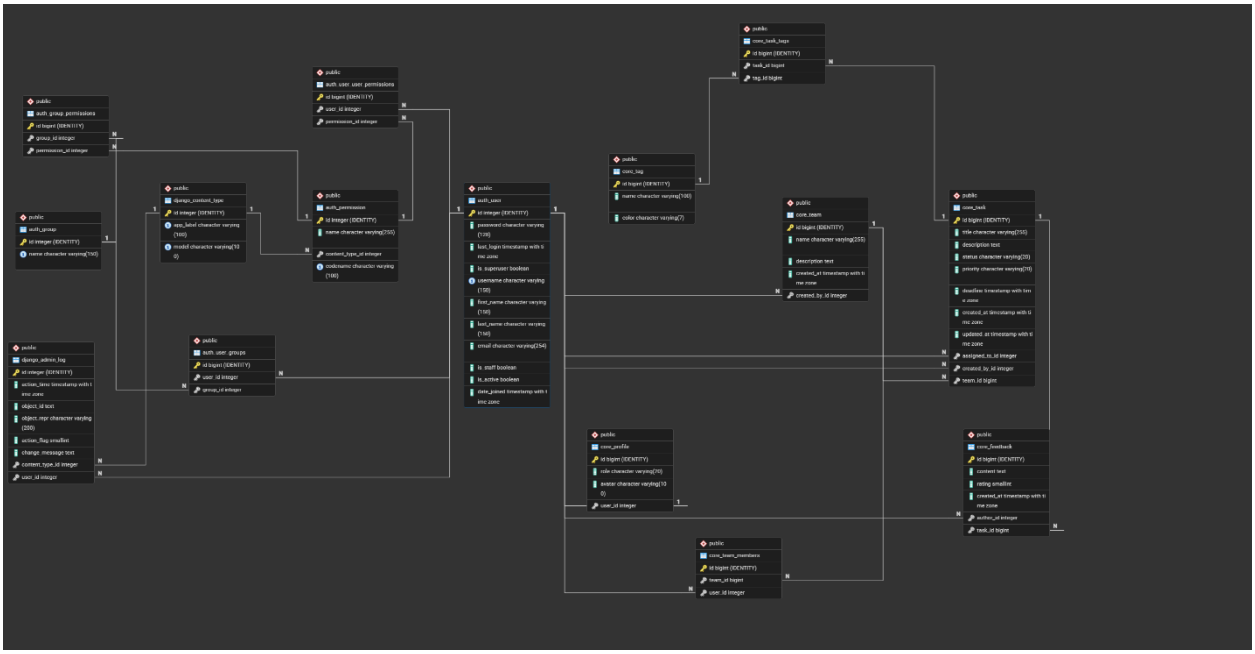




Database Schema

The application has six core models with the following relationships:

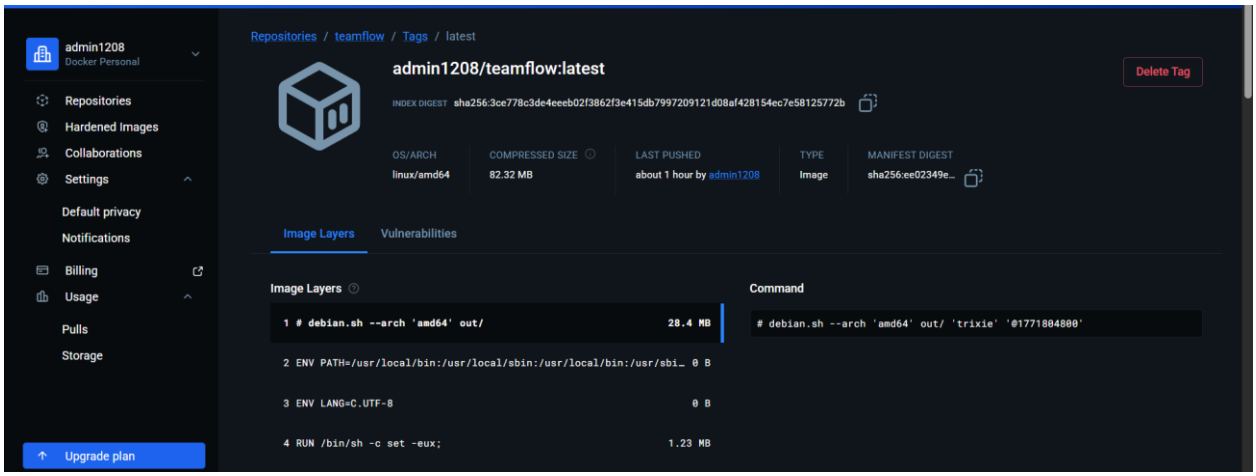
- User ↔ Profile — one-to-one (auto-created via Django signals)
- Team → User — many-to-many for members, foreign key for creator
- Task → Team — many-to-one (FK); Task → User — FK for assignee/creator
- Task ↔ Tag — many-to-manyFeedback → Task and User — foreign keys



Containerization

In order to provide a consistent and reproducible deployments, the application was fully containerized in Docker. The Dockerfile employs a multi-stage build approach:

In the first stage there are installing of dependencies, requirements and compiling of assets and the final stage copies only necessary codebase or artifacts into slim Python base image, keeping the final image size about 83 MB. Additionally, non-root user is configured inside container to follow security practices and for minimizing privilege escalation risks.



The first stage (builder) installs all Python dependencies and compiles them into wheel files. The second stage (final) copies only the pre-built wheels and application code, without any build tools. The final image runs as a non-root user called appuser, which is a basic security measure to prevent privilege escalation inside the container. The entry point script waits for PostgreSQL

to be ready, runs migrations, and then starts Gunicorn with 3 workers.

```
  GNU nano 6.2                                                    docker-compose.yml
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - teamflow_net
    restart: unless-stopped

  web:
    image: admin1208/teamflow:latest
    env_file:
      - .env
    depends_on:
      - db
    networks:
      - teamflow_net
    restart: unless-stopped

  nginx:
    image: nginx:1.27-alpine
    depends_on:
      - web
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
      - static_volume:/app/staticfiles
      - media_volume:/app/media
      # Let's Encrypt (certs + webroot challenge)
      - ./certbot/www:/var/www/certbot
      - ./certbot/conf:/etc/letsencrypt
    networks:
      - teamflow_net
    restart: unless-stopped

  certbot:

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Bracket  M-Q Previous   ^B Back
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Was    M-W Next       ^F Forward
```

DOCKER COMPOSE -Service Configuration

The Docker-compose.yml defines four different services : first is db(PostgreSQL) and second is Django/Gunicorn, nginx and certbot . Web server don't expose ports directly to the host but it communicates with Nginx through the international network in Docker teamflow_net.

Volumes are also configured for 3 different purposes , such as db_data, static files and media uploads so there are these 3 volumes db_data, static_volume, media_volume. Additionally Nginx server and certbot have two additional volumes for Lets encrypt certificate files and ACME webroot directory

```
rajabboyevanvarmirzo@teamflow-prod:/opt/teamflow$ docker compose ps
NAME                 IMAGE                    COMMAND                  SERVICE   CREATED          STATUS           PORTS
teamflow-db-1        postgres:15              "docker-entrypoint.s…"   db        34 hours ago     Up 34 hours      5432/tcp
teamflow-nginx-1     nginx:1.27-alpine        "/docker-entrypoint.…"   nginx     34 hours ago     Up About an hour 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->44
3/tcp
teamflow-web-1       admin1208/teamflow:latest "/app/entrypoint.sh …"  web       About an hour ago Up About an hour 8000/tcp
rajabboyevanvarmirzo@teamflow-prod:/opt/teamflow$
```

Environment Variable Management

In the .env file all secrets and config values are stored for example Database credentials, Django secret key, allowed hosts and debug flag , also the file is listed in .gitignore and is not committed

to the repo.The web server directly read that .env in docker-compose.yml

```
  GNU nano 6.2                                                    .env
DEBUG=False
SECRET_KEY=change-me-super-secret

ALLOWED_HOSTS=teamflowa.uz,www.teamflowa.uz,34.29.238.79,localhost,127.0.0.1

DB_NAME=teamflow
DB_USER=teamflow_user
DB_PASSWORD=Tflow@2026!ProdSecure
DB_HOST=db
DB_PORT=5432

CSRF_TRUSTED_ORIGINS=http://teamflowa.uz,http://www.teamflowa.uz,http://34.29.238.79
```

Deployment configuration

1. Google Cloud VM setup

I deployed the application on Google Cloud Compute Engine VM which is running Ubuntu 22.04 LTS . First of all I installed Docker and Docker Compose plugins and created /opt/teamflow on the server and the .env file with again the secrets but directly on server.


```
  GNU nano 6.2                                                    .env
DEBUG=False
SECRET_KEY=change-me-super-secret

ALLOWED_HOSTS=teamflowa.uz,www.teamflowa.uz,34.29.238.79,localhost,127.0.0.1

DB_NAME=teamflow
DB_USER=teamflow_user
DB_PASSWORD=Tflow@2026!ProdSecure
DB_HOST=db
DB_PORT=5432

CSRF_TRUSTED_ORIGINS=http://teamflowa.uz,http://www.teamflowa.uz,http://34.29.238.79
```


After created project root , configured GCP firewall rules in order to allow inbound traffic on ports (22 SSH) , 80 HTTP , and 443 HTTPS. The VM’s static IP address (34.29.238.79) is

pointed to by domain DNS records.

VPC firewall rules Create VPC firewall rule Refresh

Filter Enter property name or value

Name	Enforcement order	Deployment scope	Priority	Description	Direction of traffic
default-allow-https	1	Global	1000	—	Ingress
default-allow-http	1	Global	1000	—	Ingress
default-allow-rdp	1	Global	65534	Allow RDP from anywhere	Ingress
default-allow-ssh	1	Global	65534	Allow SSH from anywhere	Ingress
default-allow-internal	1	Global	65534	Allow internal traffic on the default network	Ingress
default-allow-icmp	1	Global	65534	Allow ICMP from anywhere	Ingress

NGINX and GUNICORN configuration

I make the NGINX run as a docker container and act as a reverse proxy in front of GUnicorn which handles two main tasks: serving static and media files from shared volumes and forwarding other types of requests to Gunicorn server inside the web container on port 8000. Also Nginx configuration consist of two server blocks : listens on port 80 and handles ACME challenge for renewing the certificate after that redirects all traffic to HTTPS, the second listens

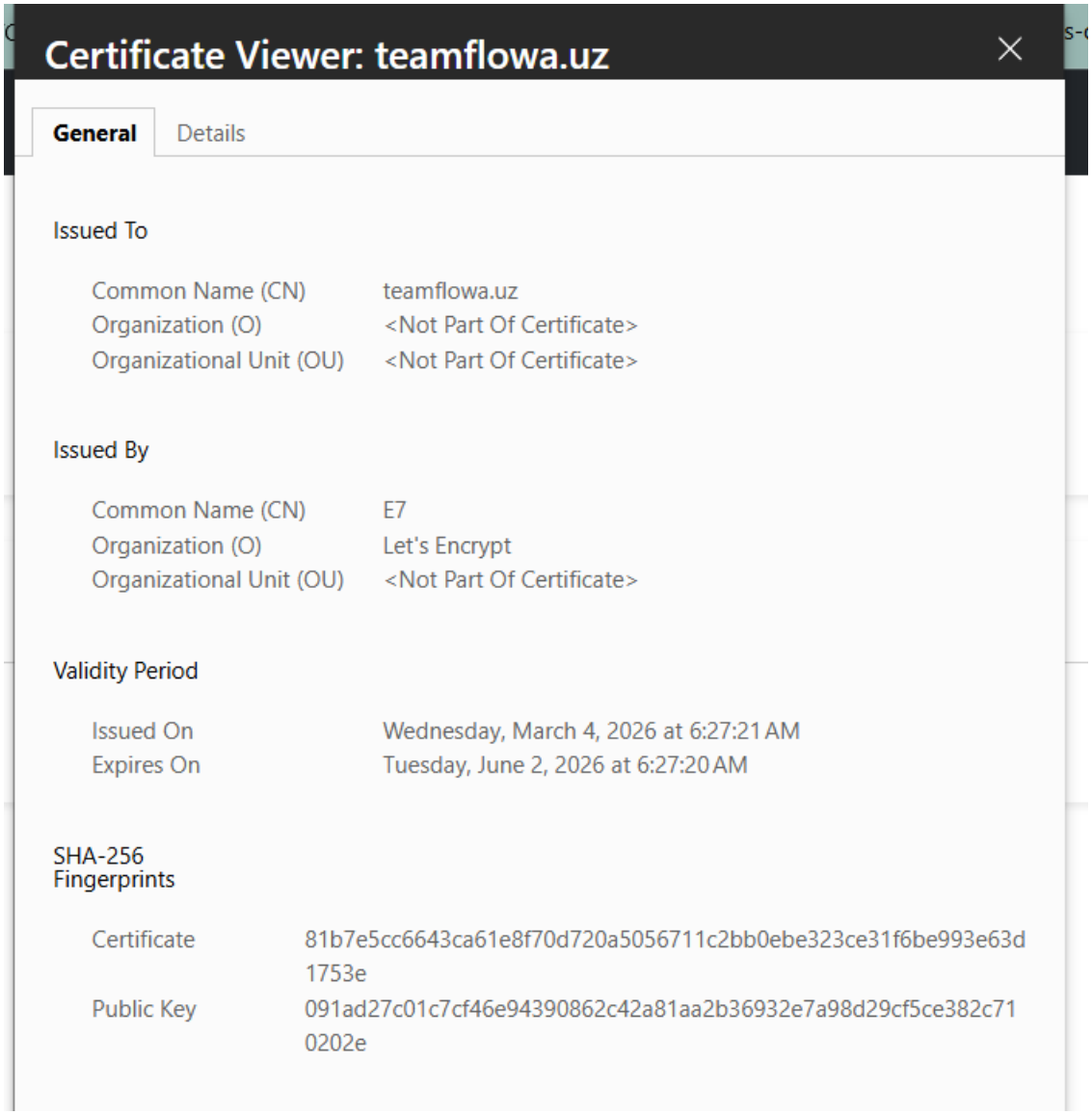
on port 443 with SSL certificate and proxies for Django upstream.

```
rajabboyevanvarmirzo@teamflow-prod:/opt/teamflow$ cat /opt/teamflow/nginx/conf.d/default.conf
upstream teamflow_web {
    server web:8000;
}

server {
    listen 80;
    server_name teamflowa.uz www.teamflowa.uz;

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}

server {
    listen 443 ssl;
    server_name teamflowa.uz www.teamflowa.uz;

    ssl_certificate /etc/letsencrypt/live/teamflowa.uz/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/teamflowa.uz/privkey.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location /static/ {
        alias /app/staticfiles/;
    }

    location /media/ {
        alias /app/media/;
    }

    location / {
        proxy_pass http://teamflow_web;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
rajabboyevanvarmirzo@teamflow-prod:/opt/teamflow$
```

SSL/HTTP Implementation

I obtain SSL certificates using Certbot with webroot method so Certboot service writes a challenge file to the shared volume and Nginx serves it at /.well-known/acme-challenge/. Once validated Lets Encrypt issued the certificate and the certificate files are mounted into Nginx container. A cron job automatically renew the certificate every day at 3 am and reload Nginx

after renewal.



Domain Configuration

The domain teamflowa.uz was configured at the DNS provider with two A records pointing to the GCP VM's external IP (34.29.238.79): one for the root domain (@) and one for www. Initially there was a second A record pointing to the old server (185.196.212.52), which caused Let's Encrypt validation to fail because some validation requests hit the wrong server. Removing the old record resolved the issue.

CI/CD pipeline

I implemented a CI/CD pipeline with Github Actions in .github/workflows/deploy.yml where the YAML file triggers automatically on every push to the main branch and runs test-only checks on

pull requests. The pipeline has three jobs that run in sequence : test>>build>>deploy

JOB 1

This job spins up a PostgreSQL service container and runs test suite against it :

steps are:

checkout code

setup Python 3.11

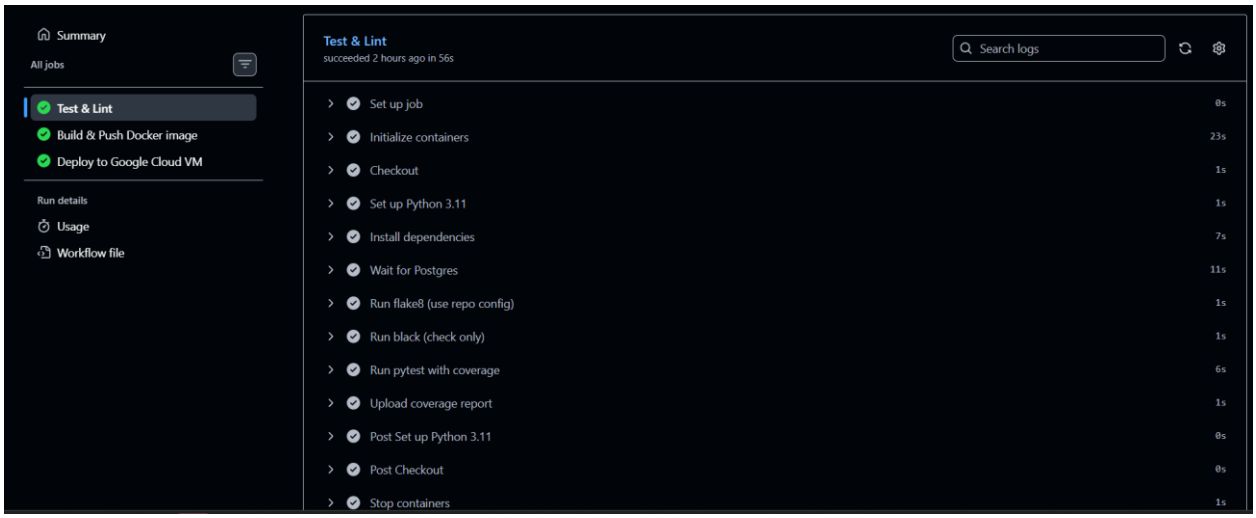
Install dependencies

run flake8 for style checking

Run black for format checking

Run Pytest with coverage

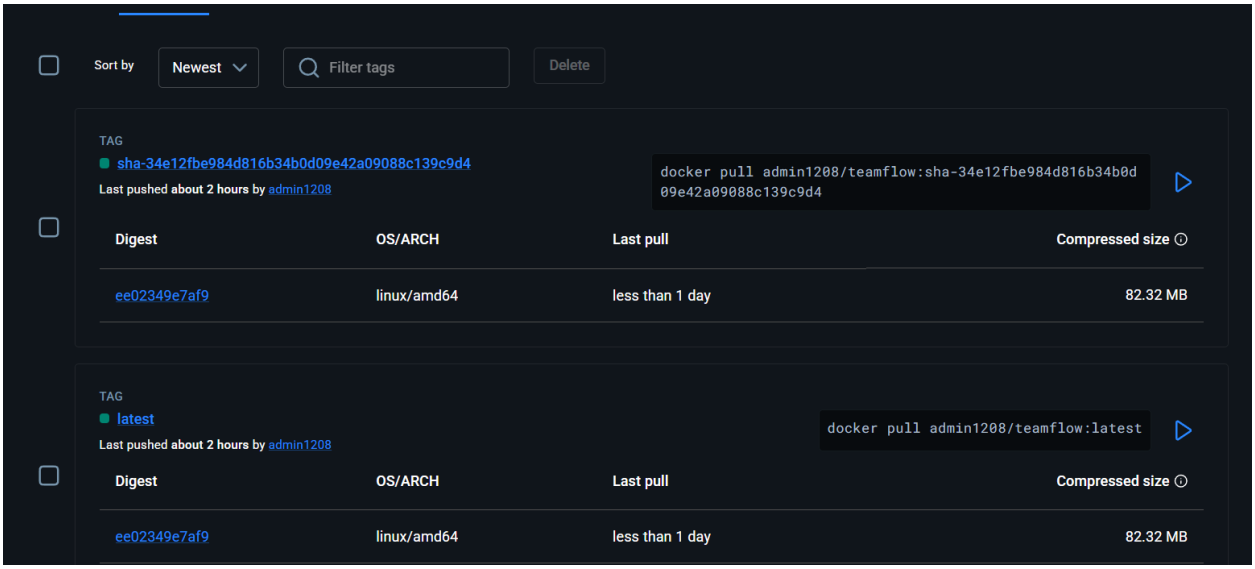
This job runs on all pushes and pull requests, which means if something wrong format or style or bad code is caught before it reaches the server.



JOB 2

This job only runs on pushes to main, after the test job passes. It builds the Docker image using the production Dockerfile and pushes it to Docker Hub under admin1208/teamflow with two tags: latest and the full commit SHA. Using the commit SHA tag makes it possible to roll back to

any specific version if needed.



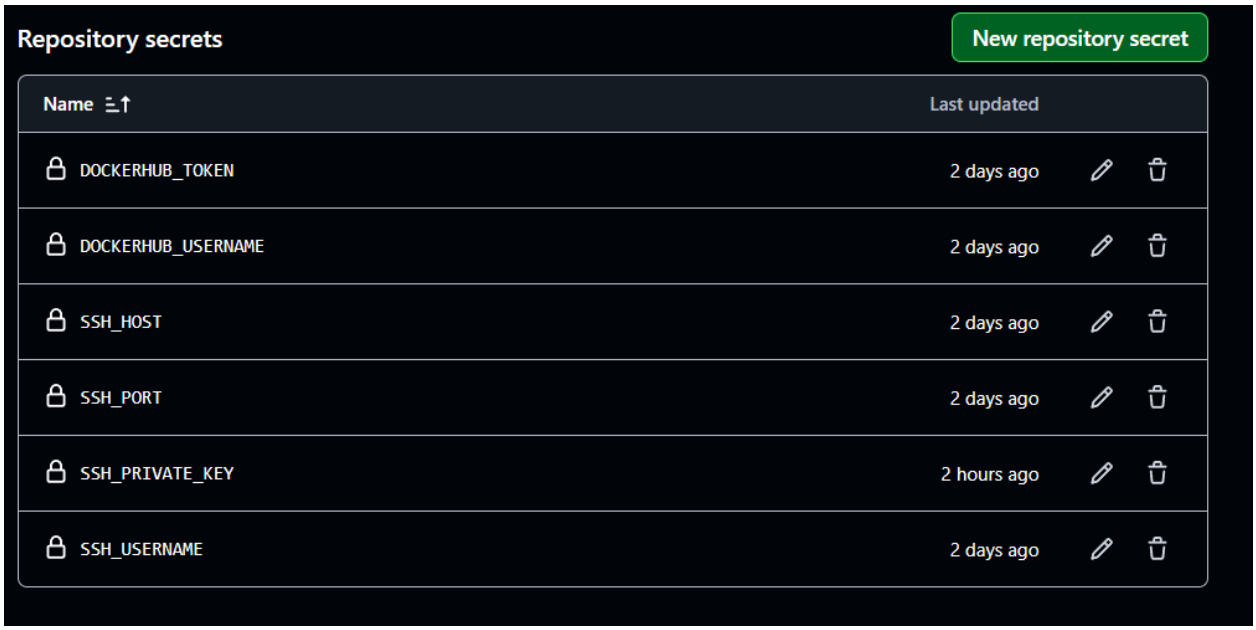
JOB 3

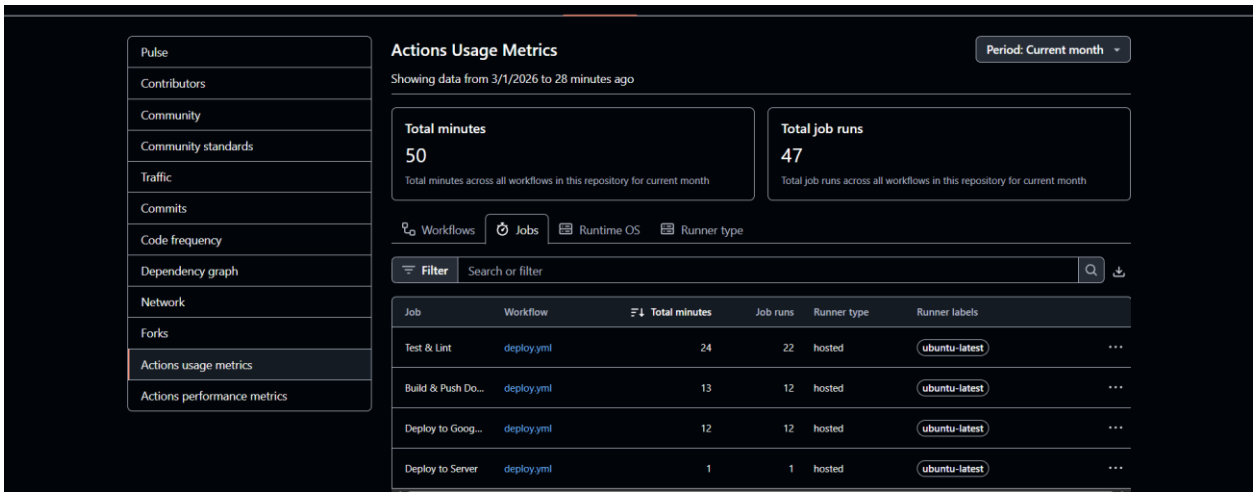
The deploy job connects to the server via SSH using the appleboy/ssh-action. It pulls the new image, restarts the web service, runs database migrations, collects static files, and prunes old Docker images. After deployment, a health check polls https://teamflowa.uz/health/ up to 30 times with 2-second intervals. If the endpoint returns 200 OK, the deployment is marked successful.

SECRETS Management

Five GitHub Secrets are configured in the repository: DOCKERHUB_USERNAME, DOCKERHUB_TOKEN, SSH_HOST, SSH_USERNAME, and SSH_PRIVATE_KEY. The SSH key pair was generated on the server with ed25519 algorithm. The public key is in

~/.ssh/authorized_keys on the server, and the private key is stored only in GitHub Secrets — it never touches the repository.





Challenges

During Development and Deployment I face several challenges : Challenge 1 – Duplicate DNS A record , after I bought a domain from ahost.com it was registered to DNS automatically with the companys info web page ip address during a creation I oversee and created an A rcord with my VM’s ip and fail with a 404 error. The solution was a deleting the old record and recreate mine and wait 12 hours to re-registration.

Challenge 2 – SSH auth in CI/CD

The GitHub Actions deploy job was failing with 'ssh: unable to authenticate'. The problem was

that the server had no authorized_keys file at all, so no SSH key was trusted. The fix was to generate a new ed25519 key pair on the server, add the public key to ~/.ssh/authorized_keys with correct permissions (600), and save the private key in GitHub Secrets. After this, the pipeline connected successfully.

FUTURE Improvements

If I were to continue developing this project, I would add Redis for session caching and Celery for asynchronous tasks such as sending email notifications when tasks are assigned. I would also set up a staging environment that deploys from a dev branch before anything reaches production. Additionally, I would configure HSTS headers and Content Security Policy in Nginx to improve security.

LINKS:

1) Live Application HTTPS : https://www.teamflowa.uz/dashboard/

2) GitHub Repository : https://github.com/Rajabboev/Teamflow_Django_app.git

3) Docker Hub Repository :

https://hub.docker.com/repository/docker/admin1208/teamflow/general

TEST USER CREDENTIALS

USERNAME: Administrator

Password: *PasswordAdmin*

Role: Team Lead Frontend – full access

GIT HUB COMMIT SCREENSHOTS:

Teamflow_Django_app Public

Pin Watch 0

390352d 1 Branch 0 Tags Go to file t Code

00016577 Initial commit: Teamflow Django project 390352d · 5 days ago 1 Commit

core	Initial commit: Teamflow Django project	5 days ago
deploy	Initial commit: Teamflow Django project	5 days ago
docs	Initial commit: Teamflow Django project	5 days ago
nginx	Initial commit: Teamflow Django project	5 days ago
teamflow	Initial commit: Teamflow Django project	5 days ago
tests	Initial commit: Teamflow Django project	5 days ago
.dockerignore	Initial commit: Teamflow Django project	5 days ago

Teamflow_Django_app Public

Pin | Watch 0

1415a9d | 1 Branch | 0 Tags | Go to file t | Code

00016577 Add add-member view and CI/CD pipeline — 1415a9d · 2 days ago — 2 Commits

.github/workflows	Add add-member view and CI/CD pipeline	2 days ago
core	Add add-member view and CI/CD pipeline	2 days ago
deploy	Initial commit: Teamflow Django project	5 days ago
docs	Initial commit: Teamflow Django project	5 days ago
nginx	Initial commit: Teamflow Django project	5 days ago
teamflow	Initial commit: Teamflow Django project	5 days ago
tests	Initial commit: Teamflow Django project	5 days ago
.dockerignore	Initial commit: Teamflow Django project	5 days ago
.env.example	Initial commit: Teamflow Django project	5 days ago
.gitignore	Initial commit: Teamflow Django project	5 days ago

Commit c862ef5 — Browse files

00016577 committed 2 days ago · 0 / 3

Fix flake8 warnings and settings imports

main — 1 parent 6cb58da commit c862ef5

6 files changed +4 -8 lines changed — Search within code

core/__init__.py -1

```
@@ -1,2 +1 @@
# Core app package
-
```

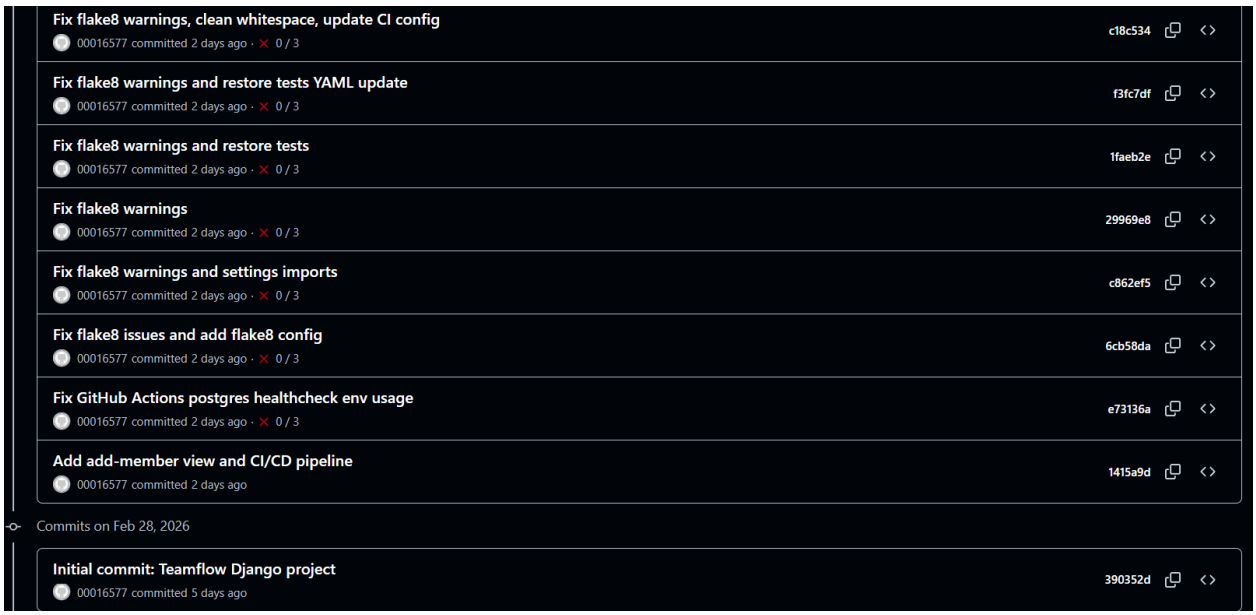
```
# Core app package
```

core/management/__init__.py -1

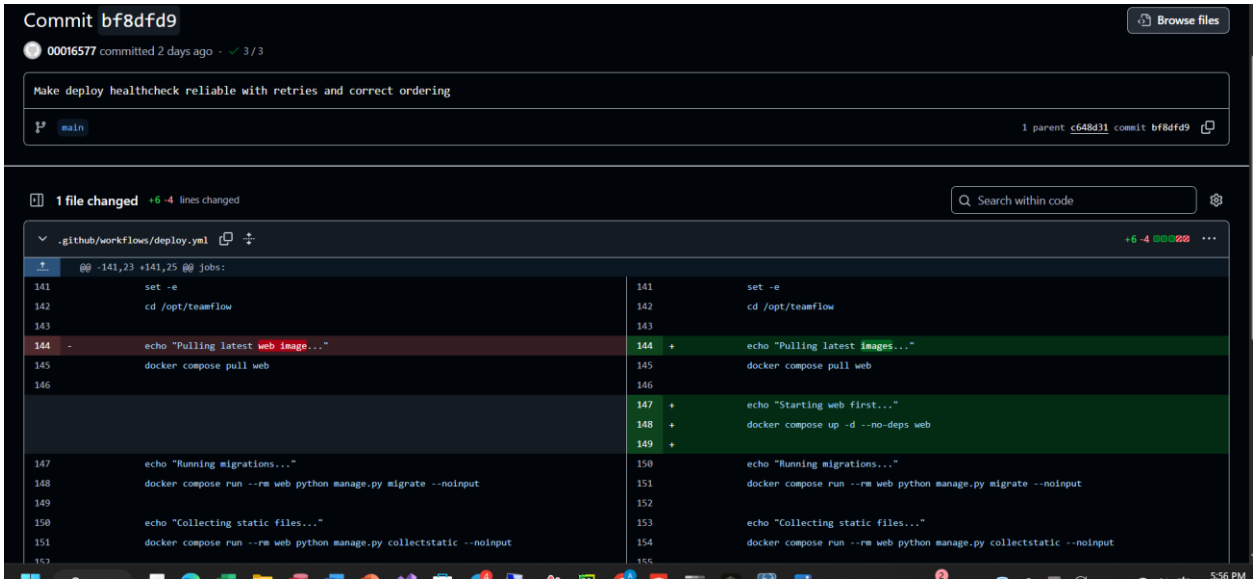
```
@@ -1,2 +1 @@
# Core management package
-
```

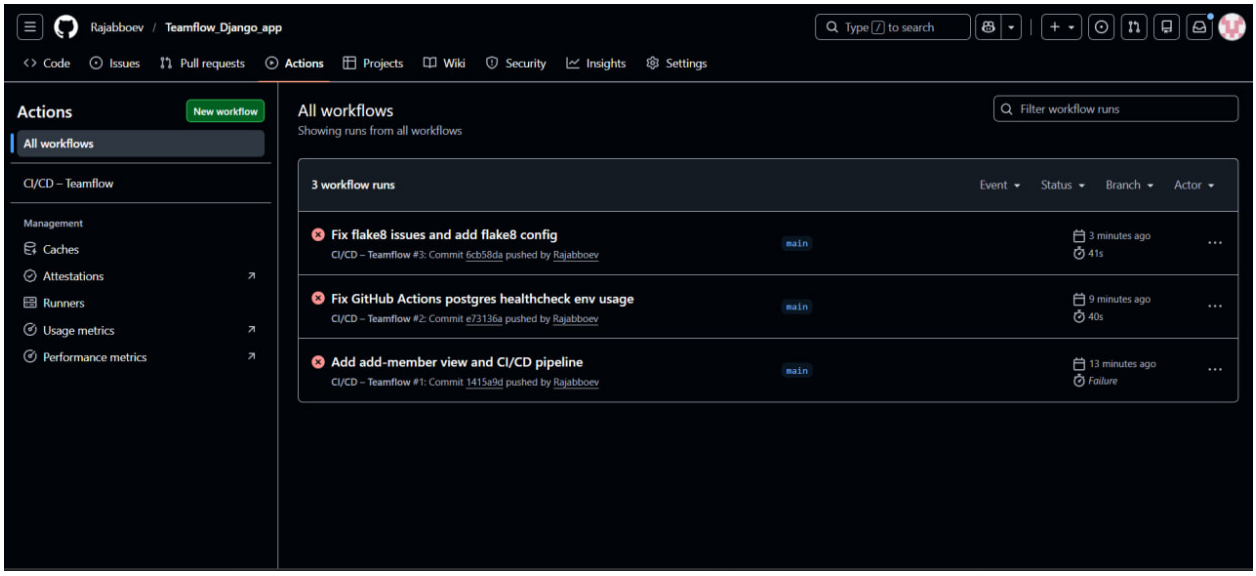
```
# Core management package
```

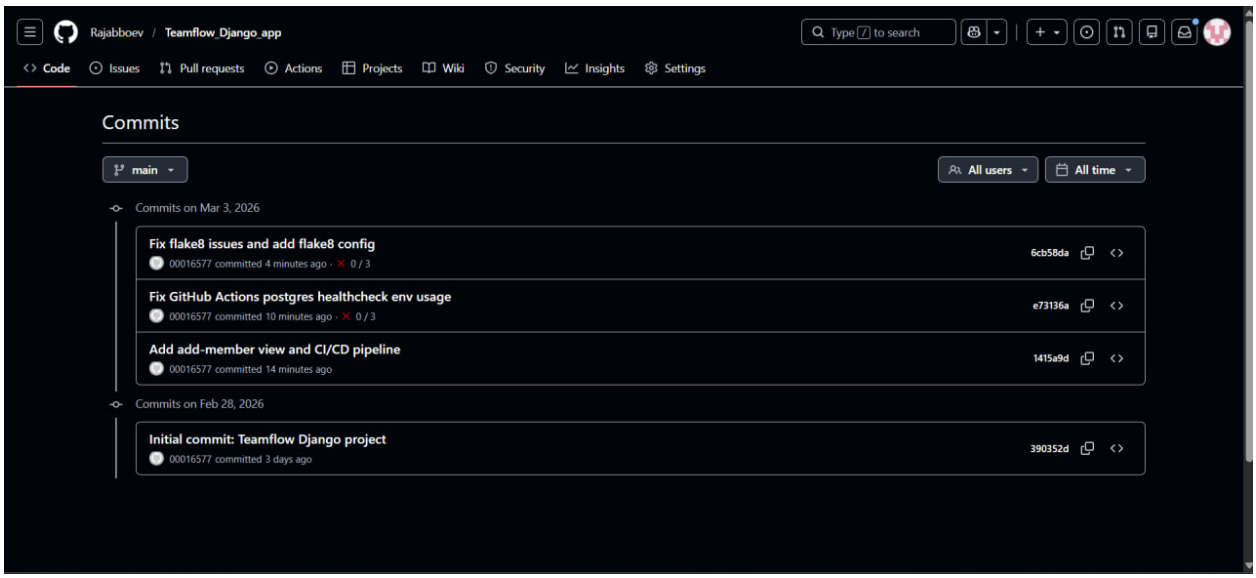
core/management/commands/__init__.py -1

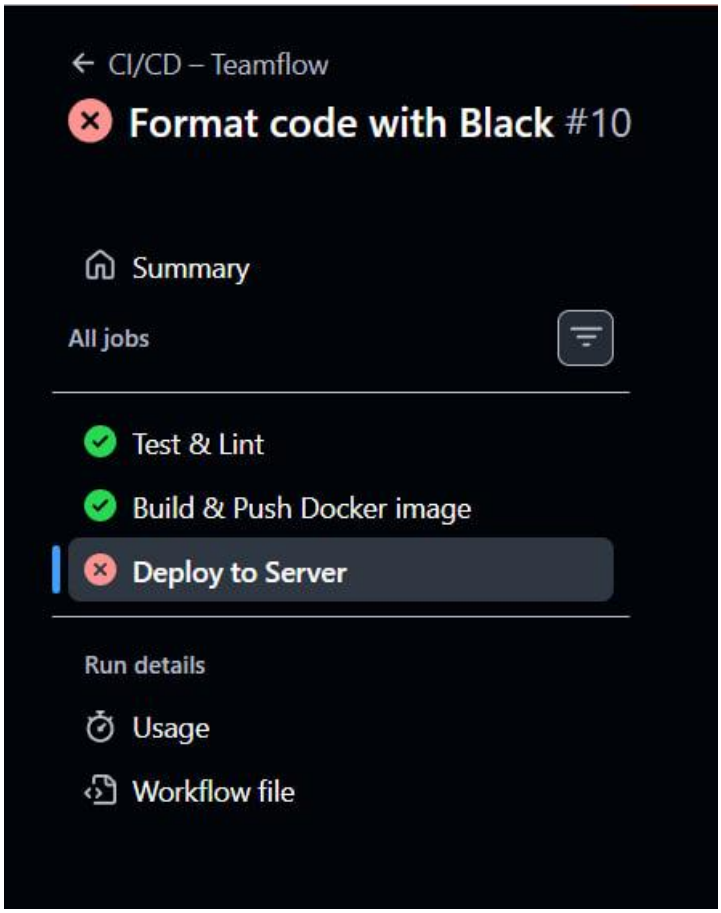


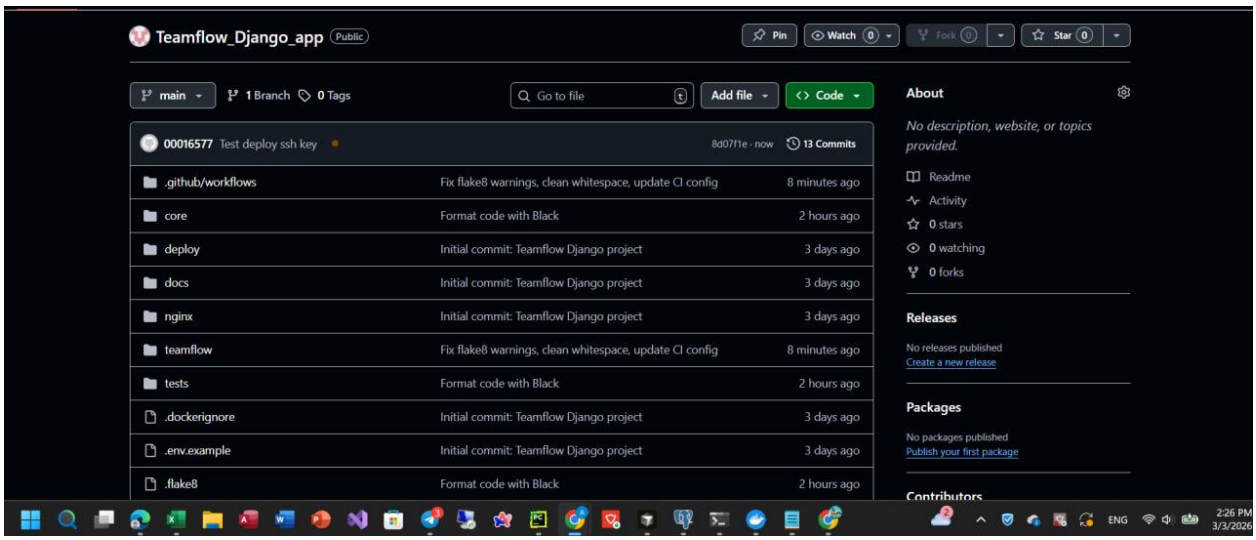
For many commits are to fix flake so one of them is provided

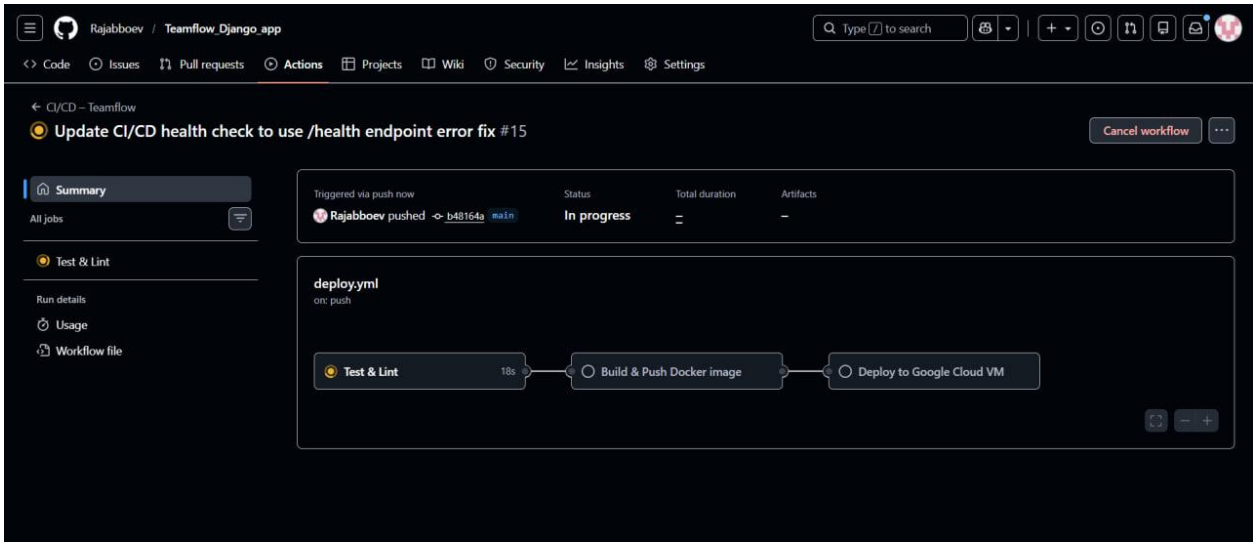


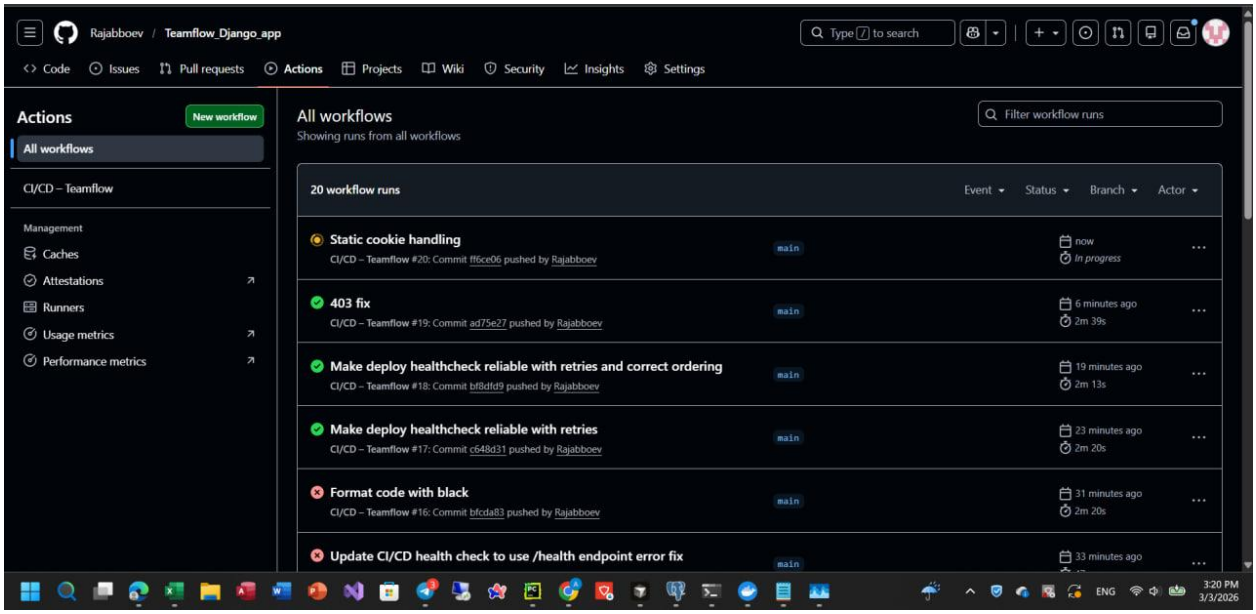



Rajabboev / Teamflow_Django_app
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Commits
main
All users
All time
Commits on Mar 3, 2026
Fix flake8 issues and add flake8 config
00016577 committed 4 minutes ago · 0 / 3
6cb58da
Fix GitHub Actions postgres healthcheck env usage
00016577 committed 10 minutes ago · 0 / 3
e73136a
Add add-member view and CI/CD pipeline
00016577 committed 14 minutes ago
1415a9d
Commits on Feb 28, 2026
Initial commit: Teamflow Django project
00016577 committed 3 days ago
390352d


CI/CD – Teamflow
Format code with Black #10
Summary
All jobs
Test & Lint
Build & Push Docker image
Deploy to Server
Run details
Usage
Workflow file


Teamflow_Django_app
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
00016577 Test deploy ssh key
8d07f1e · now
13 Commits
.github/workflows
Fix flake8 warnings, clean whitespace, update CI config
8 minutes ago
core
Format code with Black
2 hours ago
deploy
Initial commit: Teamflow Django project
3 days ago
docs
Initial commit: Teamflow Django project
3 days ago
nginx
Initial commit: Teamflow Django project
3 days ago
teamflow
Fix flake8 warnings, clean whitespace, update CI config
8 minutes ago
tests
Format code with Black
2 hours ago
.dockerignore
Initial commit: Teamflow Django project
3 days ago
.env.example
Initial commit: Teamflow Django project
3 days ago
.flake8
Format code with Black
2 hours ago
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors


Rajabboev / Teamflow_Django_app
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
CI/CD – Teamflow
Update CI/CD health check to use /health endpoint error fix #15
Cancel workflow
Summary
All jobs
Test & Lint
Run details
Usage
Workflow file
Triggered via push now
Rajabboev pushed b48164a main
Status
In progress
Total duration
Artifacts
deploy.yml
on: push
Test & Lint
18s
Build & Push Docker image
Deploy to Google Cloud VM


Rajabboev / Teamflow_Django_app
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Actions
New workflow
All workflows
CI/CD – Teamflow
Management
Caches
Attestations
Runners
Usage metrics
Performance metrics
All workflows
Showing runs from all workflows
Filter workflow runs
20 workflow runs
Event
Status
Branch
Actor
Static cookie handling
CI/CD – Teamflow #20: Commit ff6ce06 pushed by Rajabboev
main
now
In progress
403 fix
CI/CD – Teamflow #19: Commit ad75e27 pushed by Rajabboev
main
6 minutes ago
2m 39s
Make deploy healthcheck reliable with retries and correct ordering
CI/CD – Teamflow #18: Commit bf8dfd9 pushed by Rajabboev
main
19 minutes ago
2m 13s
Make deploy healthcheck reliable with retries
CI/CD – Teamflow #17: Commit c648d31 pushed by Rajabboev
main
23 minutes ago
2m 20s
Format code with black
CI/CD – Teamflow #16: Commit bfcda83 pushed by Rajabboev
main
31 minutes ago
2m 20s
Update CI/CD health check to use /health endpoint error fix
main
33 minutes ago

DSCC main Current File

Project

DSCC C:\Users\admin\Desktop\DSCC
- .github
- .pytest_cache
- .venv library root
- core
- deploy
- docs
  - pg_hba_snippet.conf
- media
- nginx
- teamflow
- tests
- .dockerignore
- .env.example
- .flake8

base.html README.md pg_hba_snippet.conf

```
55                  <ul class="nav flex-column">
73      <div class="alert alert-{% if message.tags == 'error' %}danger{% else %}{{ message.tags|de
74          {{ message }}
75          <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Cl
76      </div>
77      {% endfor %}
78  </div>
79  {% endif %}
80  {% block content %}{% endblock %}
81  main>
82
83
84
85  ass="mt-auto py-2 bg-light text-center text-muted small">
86  flow
87
```

html › body.d-flex.flex-column.min-vh-100 › div.container-fluid.flex-grow-1 › div.row › nav.col-md-2.d-md-block.bg-light.sidebar.collapse.py-3

Terminal Local

```
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
(.venv) PS C:\Users\admin\Desktop\DSCC> git pull origin main --rebase
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 8 (delta 2), reused 1 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (8/8), 1.97 KiB | 223.00 KiB/s, done.
From https://github.com/Rajabboev/Teamflow_Django_app
 * branch            main       -> FETCH_HEAD
   ff6ce06..3121b01  main       -> origin/main
Successfully rebased and updated refs/heads/main.
(.venv) PS C:\Users\admin\Desktop\DSCC> git push origin main
```

DSCC › core › templates › core › base.html 65:26 CRLF UTF-8 4 spaces Python 3.11 (DSCC)